HOLDBUFFER

AIR FORCE RESEARCH LABORATORY

December 16, 2025

Jay Convertino

Contents

1 Usage **2**
- 1.1 Introduction . . . 2
- 1.2 Dependencies . . . 2
- 1.3 In a Project . . . 2

2 Architecture **3**

3 Building **3**
- 3.1 fusesoc . . . 3
- 3.2 Source Files . . . 4
  - 3.2.1 fusesoc_info File List . . . 4
- 3.3 Targets . . . 4
  - 3.3.1 fusesoc_info Targets . . . 4
- 3.4 Directory Guide . . . 4

4 Simulation **5**
- 4.1 iverilog . . . 5
- 4.2 cocotb . . . 5

5 Module Documentation **6**
- 5.1 holdbuffer . . . 7
- 5.2 tb_cocotb-py . . . 10
- 5.3 tb_cocotb-v . . . 13

1 Usage

1.1 Introduction

This core is used for pipelining axi-x interfaces alonge with others such as FIFOS. It is made to be flexable and the inclusion of the ack helps in creating a universal bus pipelining core. This decouples the ready and registers all data. There will be latency. Similar to a skid buffer in concept.

1.2 Dependencies

The following are the dependencies of the cores.

- fusesoc 2.X
- iverilog (simulation)
- cocotb (simulation)

1.3 In a Project

This can be used to pipeline any bus to bus interface. It mimicks the axi streaming one for one, but most others are similar enough to plug in without issue. The ack is added for this interfaces that require it and allows more flexable options vs just axi streaming (which includes standard AXI channels). If no timeout or enable is needed set timeout to 0 and enable to 1 and they will be ignored. If ack is needed the ACK_ENABLE parameter must be set to 1.

Adding the core to a fusesoc project, outside of adding the verilog module to your code, requires it is added as dependency. Example:

```
dep:
  depend:
    - ">=AFRL:simple:holdbuffer:1.0.0"
```

Module instantiation:

```
holdbuffer #(
  .BUS_WIDTH(BUS_WIDTH)
) inst_holdbuffer (
  .clk(clk),
  .rstn(rstn),
  .timeout(timeout),
  .enable(enable),
  .s_data(s_data_tdata),
  .s_data_last(s_data_tlast),
  .s_data_valid(s_data_tvalid),
```

```
    .s_data_ready(s_data_tready),
    .s_data_ack(s_data_ack),
    .m_data(m_data_tdata),
    .m_data_last(m_data_tlast),
    .m_data_valid(m_data_tvalid),
    .m_data_ready(m_data_tready),
    .m_data_ack(m_data_ack)
  );
```

2 Architecture

This core can be used with any bus. Its data width is dictated in bytes and has the ability to be forced out of hold with a timeout signal. It also has a enable to turn off all outputs of the core to mask out data. The core will buffer data in normal operation and its best case latency is one clock cycle. Its worst case is two when ready is not active (we are not ready). Usage is similar to a axi streaming interface.

- **holdbuffer** decouple ready signal for pipelining.

Please see 5 for more information.

3 Building

The HOLDBUFFER core is written in Verilog 2001. They should synthesize in any modern FPGA software. The core comes as a fusesoc packaged core and can be included in any other core as a dependency. Be sure you have meet the dependencies listed in the previous section. This core also has a target called lint for verilog linting useing verible-verilog-lint.

3.1 fusesoc

Fusesoc is a system for building FPGA software without relying on the internal project management of the tool. Avoiding vendor lock in to Vivado or Quartus. These cores, when included in a project, can be easily integrated into targets created based upon the end developer needs. The core by itself is not a part of a system and should be integrated into a fusesoc based system. Simulations are setup to use fusesoc and are a part of its targets.

3.2 Source Files

3.2.1 fusesoc_info File List

- src
  - src/holdbuffer.v
- tb
  - tb/tb_holdbuffer.v
- tb_cocotb
  - 'tb/tb_cocotb.py': 'file_type': 'user', 'copyto': '.'
  - 'tb/tb_cocotb.v': 'file_type': 'verilogSource'

3.3 Targets

3.3.1 fusesoc_info Targets

- default

  Info: Default for IP intergration.
- lint

  Info: Lint with Verible
- sim_cocotb

  Info: Cocotb unit tests

3.4 Directory Guide

Below highlights important folders from the root of the directory.

1. **docs** Contains all documentation related to this project.
   - **manual** Contains user manual and github page that are generated from the latex sources.
2. **src** Contains source files for the core
3. **tb** Contains test bench files for iverilog and cocotb
   - **cocotb** testbench files

4 Simulation

There are a few different simulations that can be run for this core. The backend used for testing is iverilog for verilog or cocotb simulations. Usually GTKWave is used to view the fst waveform output. Cocotb are the unit tests that attempt to give a pass/fail verification to the core operation.

4.1 iverilog

iverilog is used for simple test benches for quick visual verification of the core. This will autofinish after it has run up to a certain number of words have been output.

4.2 cocotb

This method allows for quick writing of test benches that actually assert and check the state of the core. These tests are much more conclusive since it will run all test vectors and generate a report if they pass or fail. All tests generate to a single fst wave file. The method of launching the tests is to use fusesoc. These have not been written to use the python runner method or makefiles. To use the cocotb tests you must install the following python libraries.

```
$ pip install cocotb
```

The targets available are listed below.

- **sim_cocotb** Standard simulation HOLDBUFFER using cocotb.

The targets above can be run with various parameters.

```
$ fusesoc run --target sim_cocotb AFRL:simple:
    ↪ holdbuffer:1.0.0
```

5 Module Documentation

- **holdbuffer** HOLDBUFFER converter
- **tb_cocotb-py** Cocotb python test routines
- **tb_cocotb-v** Cocotb verilog test bench

The next sections document the module in detail.

holdbuffer.v

AUTHORS

JAY CONVERTINO

DATES

2025/12/16

INFORMATION

Brief

Holdbuffer is used to pipeline backpressure busses.

License MIT

Copyright 2025 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

holdbuffer

```
module holdbuffer #(
parameter
integer
BUS_WIDTH
 =
8,
parameter
integer
ACK_ENABLE
 =
0
)
                                                                    (
input
```

```
 wire
 clk,
 input
 wire
 rstn,
 input
 wire
 timeout,
 input
 wire
 enable,
 input
 wire
  [BUS_WIDTH-1:0]
 s_data,
 input
 wire
 s_data_last,
 input
 wire
 s_data_valid,
 output
 wire
 s_data_ready,
 output
 wire
 s_data_ack,
 output
 wire
  [BUS_WIDTH-1:0]
 m_data,
 output
 wire
 m_data_last,
 output
 wire
 m_data_valid,
 input
 wire
 m_data_ready,
 input
 wire
 m_data_ack
)
```

Hold data till ready

Parametes

BUS_WIDTH	width of the parallel data input in bits.
ACK_ENABLE	enable ack output of all data writes, and reads.

Ports

clk input wire	global clock for the core.
rstn input wire	negative syncronus reset to clk.
timeout input wire	Active high to force core out of the hold state.
enable input wire	Active high to allow output of data. When low all output is blocked.
s_data	Input data that is BUS_WIDTH bits wide.

`input wire [BUS_WIDTH- 1:0]`	
s_data_last `input wire`	Input data has hit the end of a stream of data.
s_data_valid `input wire`	Input data is valid when 1 (high).
s_data_ready `output wire`	Core is ready for input data when 1 (high).
s_data_ack `output wire`	Output 1 when data is written to the register(s) of the core.
m_data `output wire [BUS_WIDTH- 1:0]`	Output data that is BUS_WIDTH bits wide.
m_data_last `output wire`	Output data has hit the end of a stream of data.
m_data_valid `output wire`	Output data is valid when 1 (high).
m_data_ready `input wire`	Downstream is ready to output data when 1 (high).
m_data_ack `input wire`	Downstream has registered the valid data when 1 (high).

STATES

Core has 3 states, that includes an error state.

<GET>	d1
<HOLD>	d3
<ERROR>	d0

tb_cocotb.py

AUTHORS

JAY CONVERTINO

DATES

2025/12/16

INFORMATION

Brief

Cocotb test bench

License MIT

Copyright 2024 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

FUNCTIONS

random_bool

```
def random_bool()
```

Return a infinte cycle of random bools

Returns: List

start_clock

```
def start_clock(
dut
)
```

Start the simulation clock generator.

Parameters

dut Device under test passed from cocotb test function

reset_dut

```
async def reset_dut(
dut
)
```

Cocotb coroutine for resets, used with await to make sure system is reset.

single_word

```
@cocotb.test()
async def single_word(
dut
)
```

Coroutine that is identified as a test routine. This routine tests for writing a single word, and then reading a single word.

Parameters

dut Device under test passed from cocotb.

random_ready

```
@cocotb.test()
async def random_ready(
dut
)
```

Coroutine that is identified as a test routine. This routine tests for randomized ready from the sink.

Parameters

dut Device under test passed from cocotb.

in_reset

```
@cocotb.test()
async def in_reset(
dut
)
```

Coroutine that is identified as a test routine. This routine tests if device stays in unready state when in reset.

Parameters

dut Device under test passed from cocotb.

no_clock

```
@cocotb.test()
async def no_clock(
dut
)
```

Coroutine that is identified as a test routine. This routine tests if no ready when clock is lost and device is left in reset.

Parameters

dut Device under test passed from cocotb.

tb_cocotb.v

AUTHORS

JAY CONVERTINO

DATES

2025/12/15

INFORMATION

Brief

Test bench wrapper for cocotb

License MIT

Copyright 2024 Jay Convertino

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

tb_cocotb

```
module tb_cocotb #(
parameter
BUS_WIDTH
 =
8
)
                                                                    (
input
clk,
input
rstn,
input
timeout,
input
```

```
 enable,
 output
 m_data_tvalid,
 input
 m_data_tready,
 output
  [BUS_WIDTH-1:0]
 m_data_tdata,
 output
 m_data_tlast,
 input
 s_data_tvalid,
 output
 s_data_tready,
 input
  [BUS_WIDTH-1:0]
 s_data_tdata,
 input
 s_data_tlast
)
```

Hold data till ready

Parametes

BUS_WIDTH	width of the parallel data input in bits.

Ports

clk input	global clock for the core.
rstn input	negative syncronus reset to clk.
s_data	Input data that is BUS_WIDTH bits wide.
s_data_valid	Input data is valid when 1 (high).
s_data_ready	Core is ready for input data when 1 (high).
m_data	Output data that is BUS_WIDTH bits wide.
m_data_valid	Output data is valid when 1 (high).
m_data_ready	Core is ready to output data when 1 (high).

INSTANTIATED MODULES

dut

Device under test, holdbuffer